

User Guide

Support: https://davem97.github.io/fpcc/contact

Table of Contents

1. Quickstart 3
Importing the package 3
The demo scene 3
Controls 3
HitscanShooter script 3
2. Requirements 5
Unity version and render pipeline 5
Input System compatibility 5
3. Package Overview 6
Folder structure 6
Core scripts 6
Demo-only scripts 6
4. Bullet Tracers 7
The tracer prefab 7
Adding tracers to your own weapon 7
Pooling 10
Tracer settings 10
Texture variants 11
5. Smoke Trail 13
The smoke prefab 13
Adding it to your own weapon 13
Smoke trail settings 14
6. Muzzle Effects 18
Muzzle Flash 18
Muzzle Sparks 19
Muzzle Smoke 19
7. Troubleshooting 21

1. Quickstart

Importing the package

To begin using this asset, open your Unity project, then go to Window → Package Management → Package Manager → My Assets. Select Bullet Tracers & Smoke Trails VFX, press Download, then press Import. The asset will then appear in the Project window.

The demo scene

In the Project window, go to SwitchbackGames → BulletTracers → Scenes and double-click Demo_Scene. Press play to test all features.

You may fire at the few objects in the scene and at the sky to see how tracers behave at different ranges. Shots at very close range deliberately spawn no tracer but can be tuned in the inspector of the VisualTracers script.

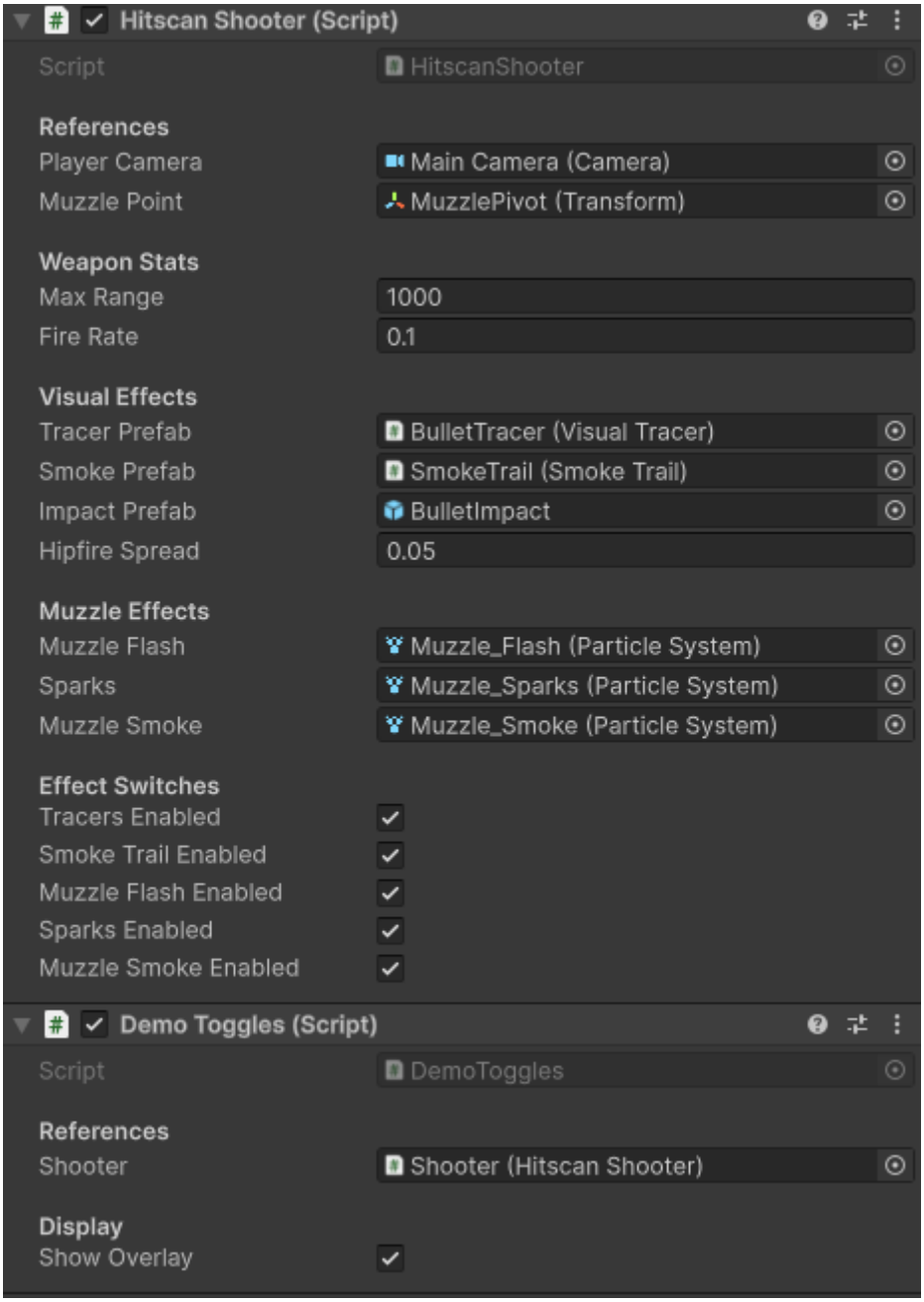
Controls

There are multiple basic controls for movement:

- W, A, S, D for standard player movement
- Space to jump
- Left mouse click to fire the weapon to spawn tracers and different VFX
- 1 to toggle the smoke trail effect
- 2 to toggle the muzzle sparks effect
- 3 to toggle the muzzle flash effect
- 4 to toggle the bullet tracer effect
- 5 to toggle the muzzle smoke effect

HitscanShooter script

This script is where all the effects run through. In the inspector, each of the effects can be added to or removed from their respective slots. Another script, DemoToggles, allows for toggling of effects during runtime so that each visual effect can be tested together or separately.



For simplicity, everything is done in a basic approach and each effect is cleanly displayed in the HitscanShooter component.

2. Requirements

Unity version and render pipeline

This asset was built and tested in Unity 6 (6000.4.10f1) using the Universal Render Pipeline. It should work in any Unity 6 version, but has not been tested in older releases.

URP is required. The tracer, smoke and muzzle effect materials use URP shaders and will appear pink in the Built-in Render Pipeline or HDRP.

The scripts themselves have no render pipeline dependency, so if you are on a different pipeline you can still use the components, but you will need to make your own materials.

The package has no external dependencies and does not require any additional packages to be installed.

Input System compatibility

The demo scripts work with both of Unity's input systems. No setup is required.

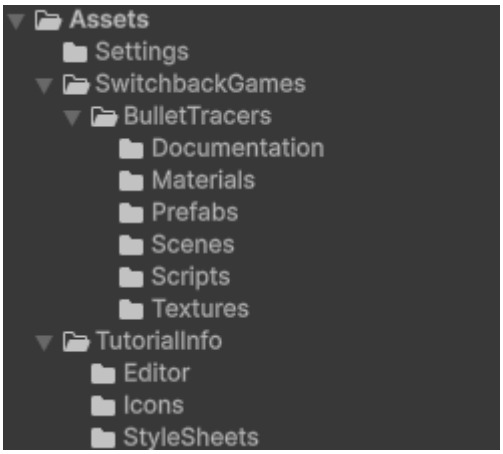
Unity projects use either the old Input Manager or the newer Input System package, and this is set in Project Settings under Player → Other Settings → Active Input Handling. The demo scripts detect which one your project uses and compile the matching code automatically.

This only affects the demo scripts (HitscanShooter, PlayerMovement, SimpleMouseLook). VisualTracer and SmokeTrail do not read input at all, so they work regardless of your setting.

3. Package Overview

Folder structure

The structure of the asset is as follows:



Core scripts

There are two core scripts, VisualTracer.cs and SmokeTrail.cs, which are the asset.

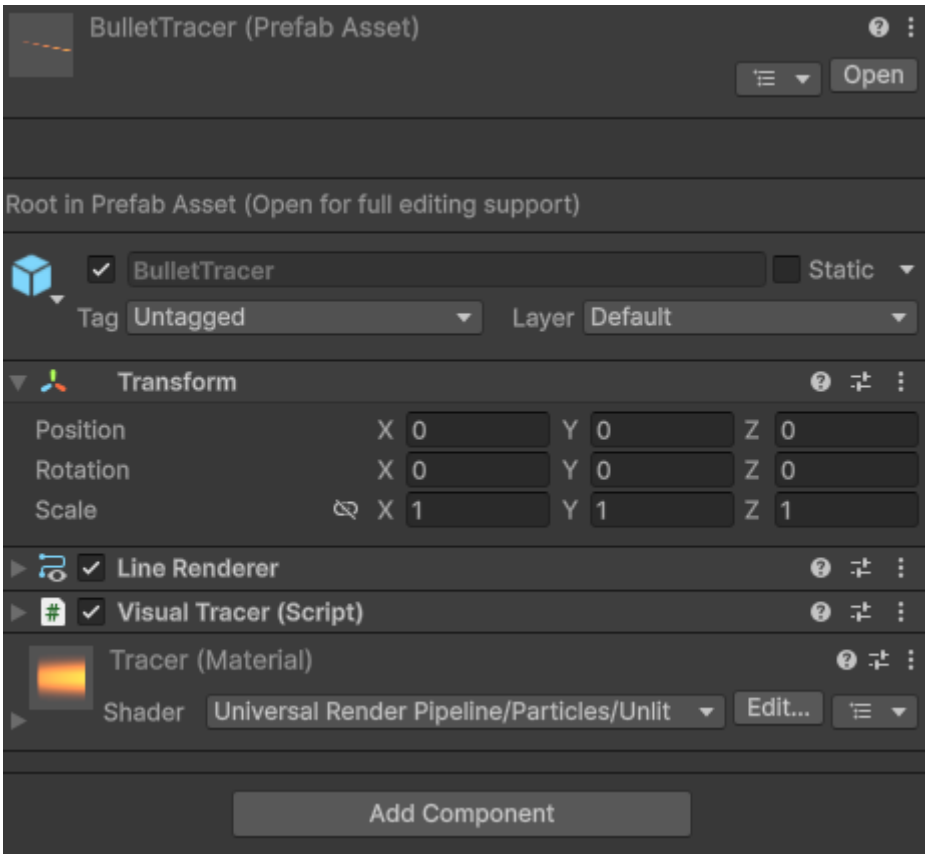
Demo-only scripts

These exist so the demo scene is playable and can be deleted. HitscanShooter.cs is an example weapon script showing how to spawn the effects, so you can use it as a reference when wiring tracers into your own weapon. PlayerMovement.cs and SimpleMouseLook.cs handle movement and looking. DemoToggles.cs handles the number key toggles.

4. Bullet Tracers

The tracer prefab

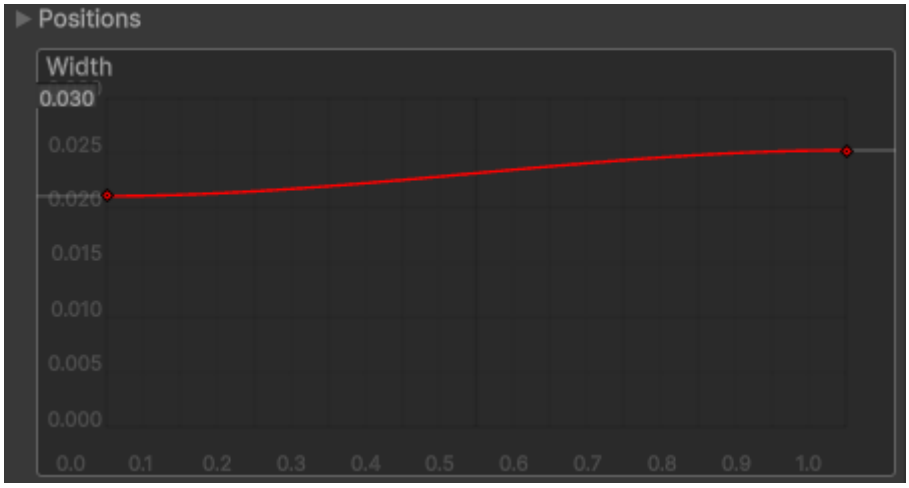
The tracer is an empty game object that uses only two components; a LineRenderer and the script VisualTracer.cs.



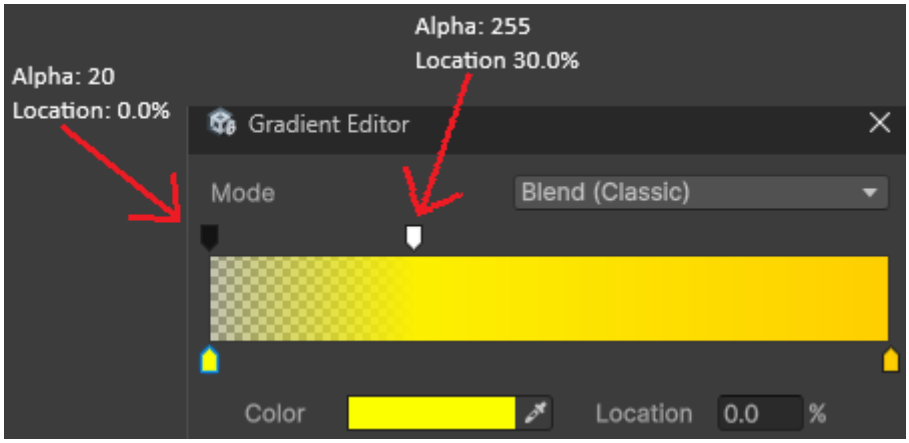
Adding tracers to your own weapon

To add bullet tracers to a project, a tracer prefab must first be created. To do this, Right click anywhere in the hierarchy → Create Empty. Rename the object (e.g., BulletTracer) → drag it into the Prefabs (or any other) folder inside the Project area. Left click on the newly created bullet tracer prefab → in the inspector settings, press “Add Component” → Search for and add the “LineRenderer” component → press “Add Component” → Search for and add the “VisualTracer” script. The default inspector values of the script can be left as they are for now.

There are a few settings to tune in the LineRenderer component to make the tracers look and function better visually. Firstly, the width of the tracer can be tuned differently and is mostly preference, but setting the start width (the red circle on the very left of the graph) to a value between 0.019 and 0.021 produces a good, thin tail, and the end width can be set to a value between 0.023 and 0.025.

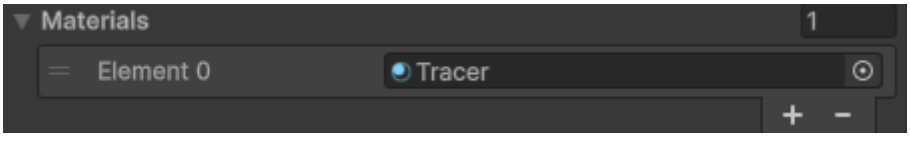


From there, the Color setting can be changed so that some transparency can be added when the tracer is first spawned and then retain its full colour immediately after, as it travels. To adjust the colour you can double click on the Color bar → left click anywhere above the colour bar inside the Gradient Editor → position the nodes to match the locations below and then adjust the alpha of both. The colour can be any shade of yellow or yellow-orange.



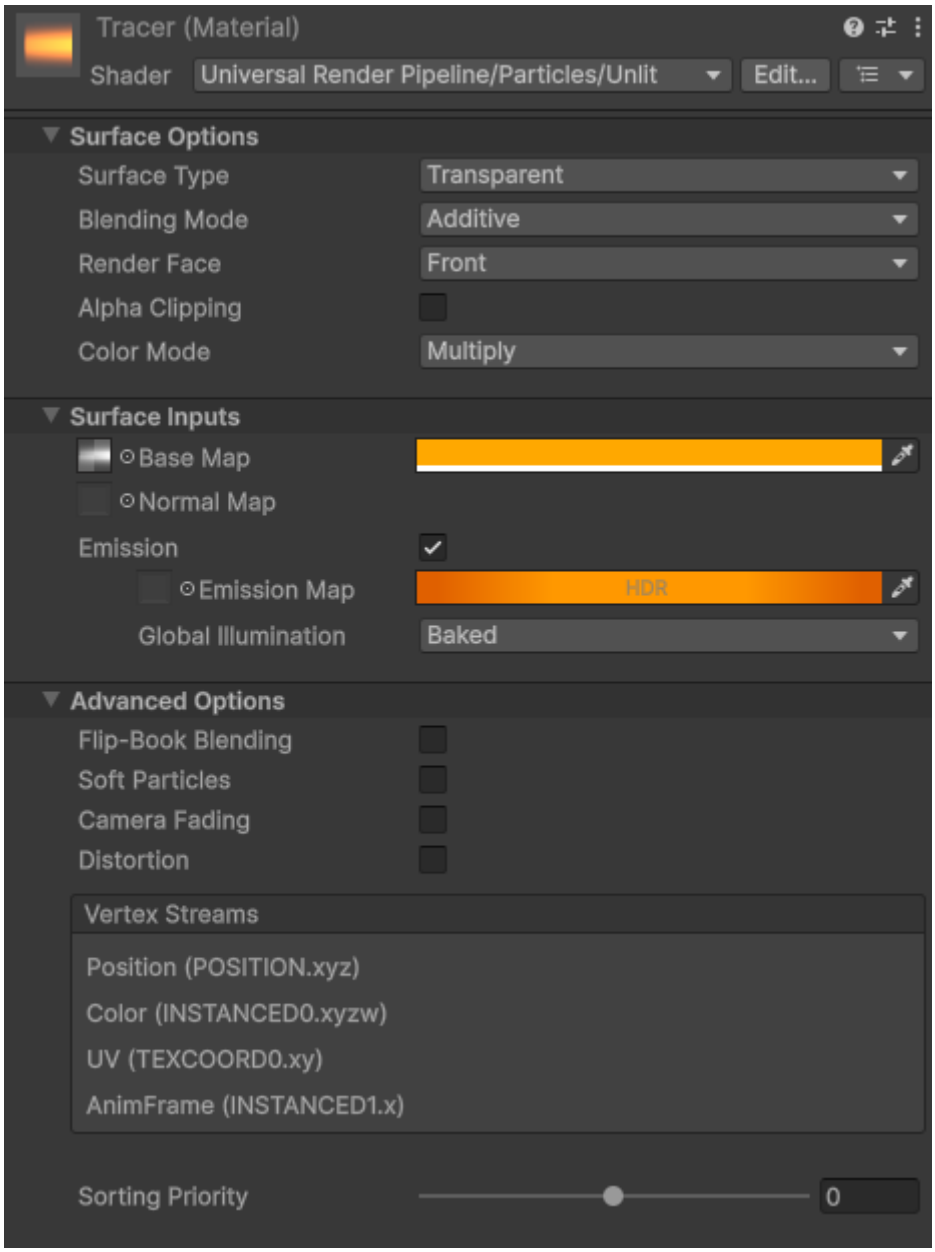
Next, the End Cap Vertices value can be changed from 0 to 6 (or even 8) to round the tip of the tracer so it looks more cylindrical rather than a flattened horizontal line. Texture Mode should be set to “Stretch”, Cast Shadows under Lighting to “Off”, and Motion Vectors under Additional Settings to “Camera Motion Only”.

For the material, in the “Element 0” slot which is under the Materials section of the LineRenderer component, you will drag and drop your tracer material there.



To create the material you can right click anywhere in the project area → Create → Material → Rename the material → Left click on the Material → Left click on the Shader dropdown at the top of the material → Change the Shader to Universal

Render Pipeline → Particles → Unlit. The material will now have additional settings to be changed which are Surface Type to “Transparent”, Blending Mode to “Additive”, Render Face can be left to “Front”, and Color Mode set to “Multiply”. Under Surface Inputs to the left of Base Map, a small empty square box can be seen. You can drag any one of the tracer textures into that slot. The texture will determine how the tracer looks. Next, click on the colour bar to the right of Base Map and change it so that it becomes yellow-orange. Then below it, tick Emission so that the box is checked, and do the same for the colour bar to the right of Emission Map and set the intensity to +1 or +2. All other settings for the material can be left as default.



After that, the tracer prefab should be ready. Adjustments can be made to the colour, textures and different shader settings if wanted.

Pooling

Firing creates a tracer every shot, so pooling avoids generating garbage during sustained fire. VisualTracer.cs supports this directly. The demo uses Unity's built-in ObjectPool, but any pooling approach works.

Set the pool up once, in Start:

```
// Tracers are pooled so firing does not create garbage every shot.
// Set this up once here, not per shot.
tracerPool = new ObjectPool<VisualTracer>(
    createFunc: () => Instantiate(tracerPrefab),
    actionOnGet: t => t.gameObject.SetActive(true),
    actionOnRelease: t => t.gameObject.SetActive(false),
    actionOnDestroy: t => Destroy(t.gameObject),
    collectionCheck: true,   // Catches double releases. Turn off for release builds
    defaultCapacity: 20,
    maxSize: 100
);

// Cached so it is not rebuilt on every shot
releaseTracerAction = t => tracerPool.Release(t);
```

Then on each shot:

```
VisualTracer tracer = tracerPool.Get();

// Set this before Initialize. Initialize can release the tracer
// straight away at point blank range
tracer.SetReleaseAction(releaseTracerAction);
tracer.Initialize(muzzlePoint, targetPoint, playerCamera);
```

SetReleaseAction must be called before Initialize. At point blank range Initialize releases the tracer immediately, and if no release action has been set it destroys itself instead of returning to the pool. Over time that drains the pool.

The tracer returns itself when it finishes travelling, so there is nothing to clean up.

Tracer settings

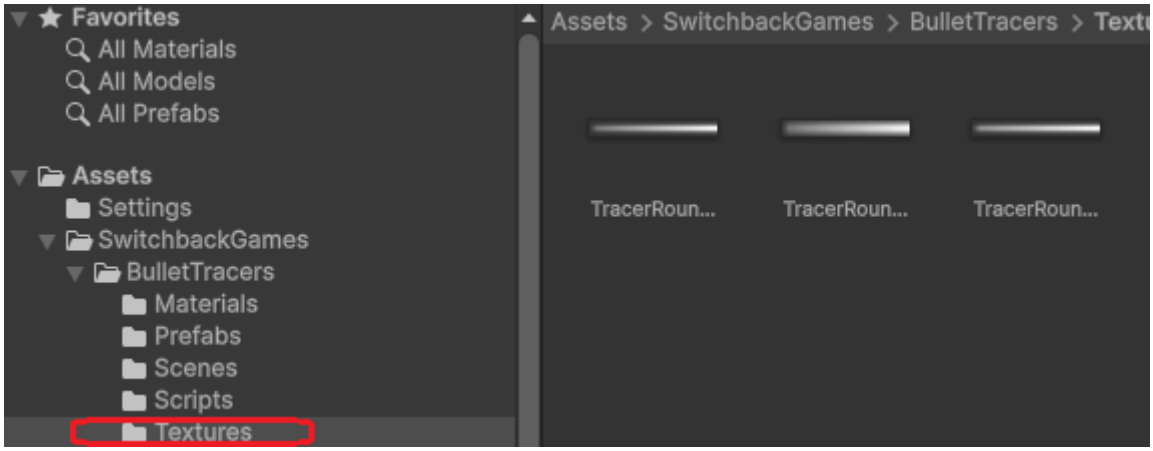
In the inspector window of the bullet tracer prefab, you will see the VisualTracer script added as a component and it will have a number of variables, most of them with tuneable values.

Group	Field	Default	What it does
Tracer settings	Tracer Length	30	How long the streak is, in metres. Capped internally at 40% of the shot distance, so close shots get a shorter streak automatically. 30 is the maximum, not the length of every tracer.
	Visual Speed	600	Overwritten on every shot by Linger Speed or the close-range calculation. The inspector value has no effect.
	Min Lifetime	0.06	Shortest time a tracer stays on screen, even if it reaches its target sooner.
	Max Lifetime	2	Hard ceiling on how long a tracer can exist. A tracer still travelling when this expires disappears rather than fading.
Close-range behaviour	Point Blank Threshold	1.9	Shots shorter than this spawn no tracer at all. There is no room for a streak to read at that distance.
Close-Range width	Width Full Distance	12	Distance at which the tracer reaches its full width.
	Width Min Distance	2	Distance at which the tracer is at its thinnest.
	Min Width Factor	0.3	How thin it gets at that distance, as a fraction of full width. At 0.3 a close-range tracer is 30% of normal thickness.
Long-Range linger	Linger Start Distance	50	Shots longer than this travel at Linger Speed. Shorter shots cross in a fixed short time instead, so they cannot be skipped between frames.
	Linger Speed	220	Travel speed for long shots, in metres per second. This is the main control for how long a tracer stays in the air. Lower values mean longer flight.
	Min Screen Pixels	2	The tracer never renders thinner than this many pixels on screen. Without it, a 0.03 unit wide line at 500 metres is a fraction of a pixel and disappears before it lands. Do not set this to 0.
Spawn variation	Min Muzzle Offset	0.3	Closest the tail of the tracer can sit to the muzzle.
	Max Muzzle Offset	3	Furthest ahead of the muzzle the tail can start. Lower this if tracers look disconnected from the gun.

• Linger Speed and Max Lifetime work against each other. A 500 m shot at 220 m/s takes about 2.3 seconds to arrive, so with Max Lifetime at 2 it is cut off shortly before it lands. If tracers vanish mid-flight, raise Max Lifetime or raise Linger Speed.
• Tracer Length and Min Screen Pixels both affect how visible a distant tracer is. Length controls how much of the streak exists; Min Screen Pixels controls whether it is thick enough to see at all.
• Width Min Distance and Point Blank Threshold are separate. Below Point Blank Threshold no tracer spawns at all; between that and Width Full Distance the tracer spawns but is thinner.

Texture variants

There are several alternative tracer textures that can be used instead of the default texture, to alter the visual appearance of the bullet tracer. These are all located in the Textures folder. These variants change the core with a strong fade towards the tail or create a broader tracer with a more diffuse glow, or less fade so the whole streak stays visible.

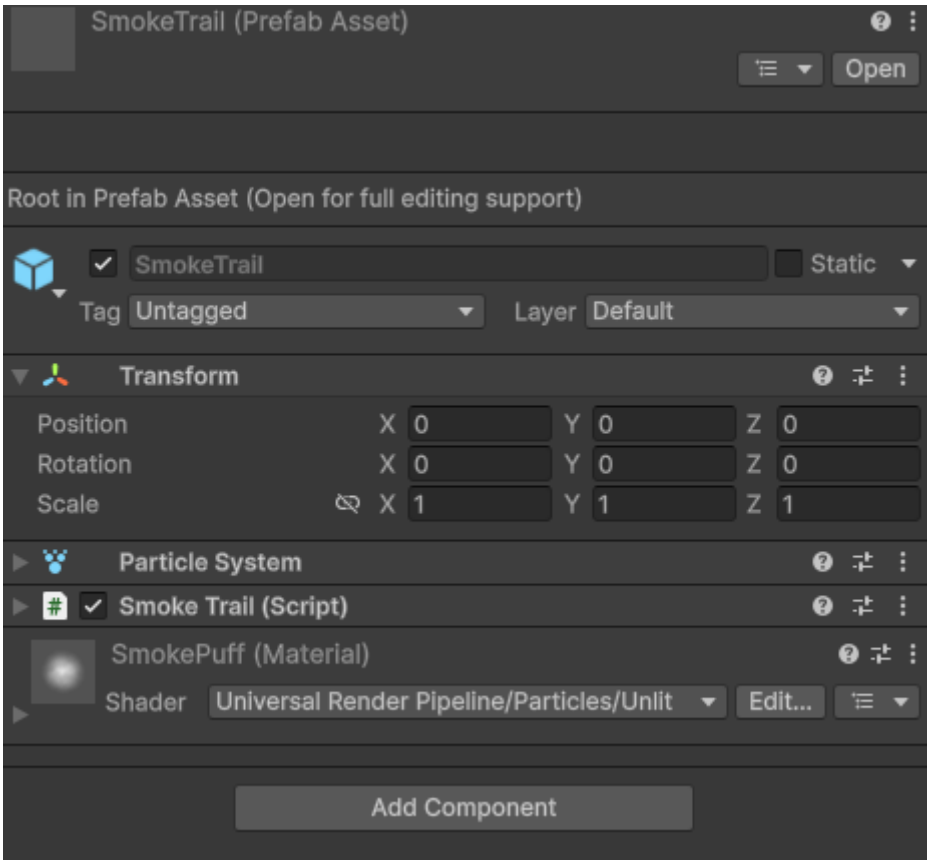


The variants can be swapped by dragging them into the Base Map slot of the tracer material.

5. Smoke Trail

The smoke prefab

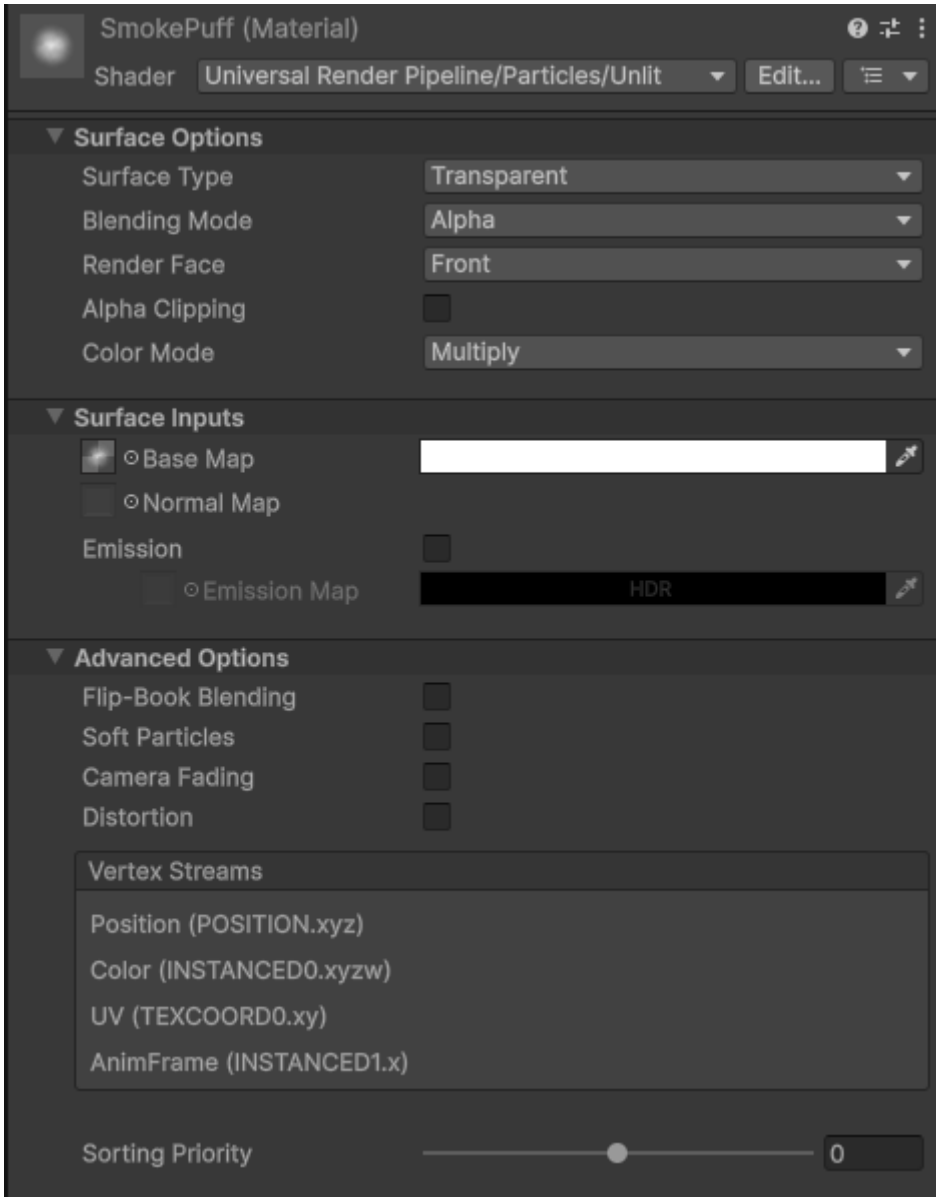
The smoke trail is an empty game object that uses only two components; a Particle System and the script SmokeTrail.cs.



Adding it to your own weapon

To add the smoke trail, you can right click anywhere in the hierarchy → Create Empty → Rename the object to SmokeTrail → In the inspector of the newly created object press “Add Component” → Search for and add a Particle System → Click “Add Component” again → Search for an add the “SmokeTrail” script. From there you can drag the smoke trail object into the “Prefabs” folder to make it a prefab and delete the object from the hierarchy. It can now be dragged into the appropriate inspector slot of the HitscanShooter game object.

To create the material for the smoke trail, the same steps from the tracer material can be followed, however in the material settings, Blending Mode will be set to “Alpha”, the colour to the right of Base Map will be default white, and the Emission box will be unchecked.

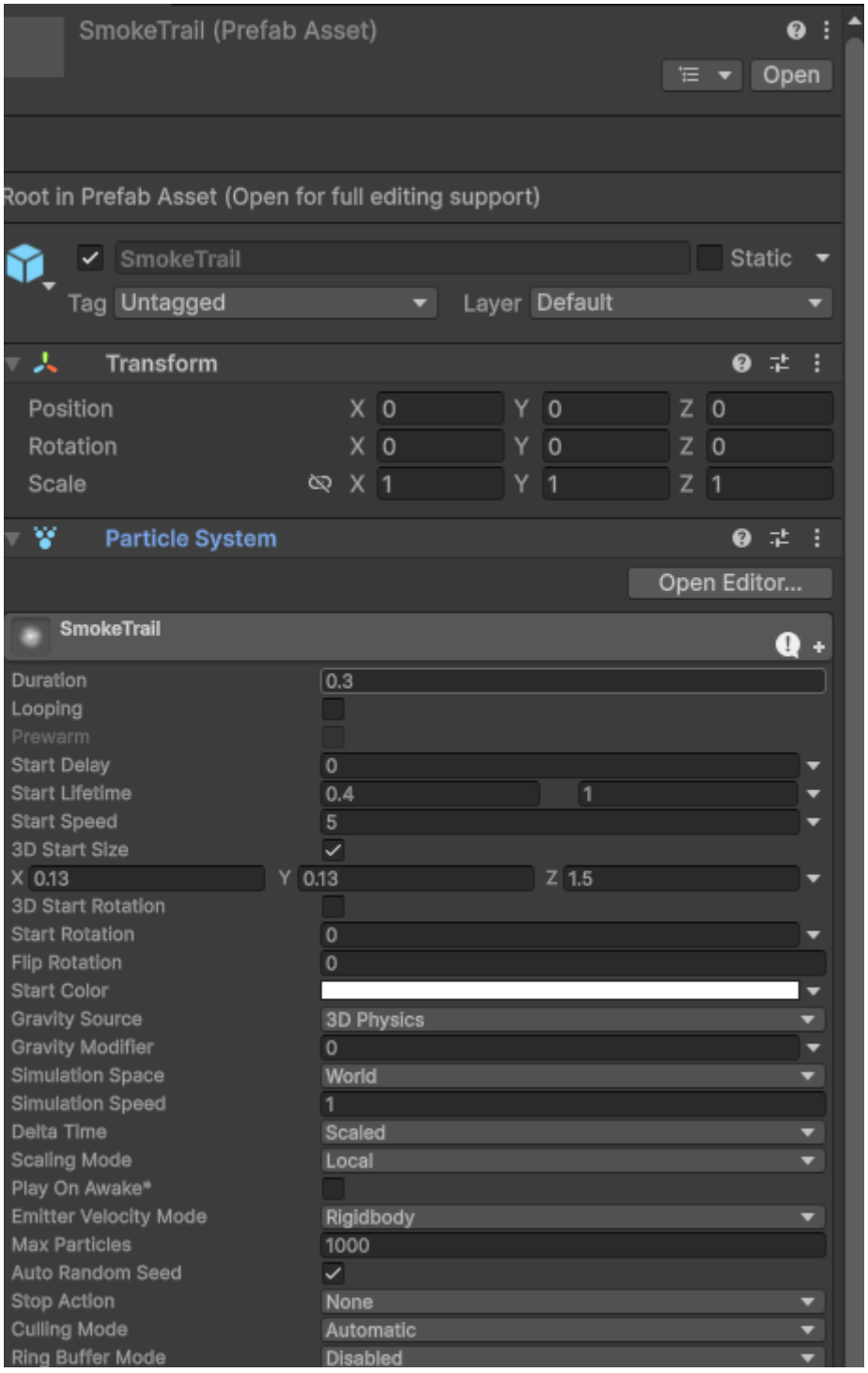


For the Base Map texture, any of the several smoke trail texture options can be dragged into the material Base Map slot under Surface Inputs.

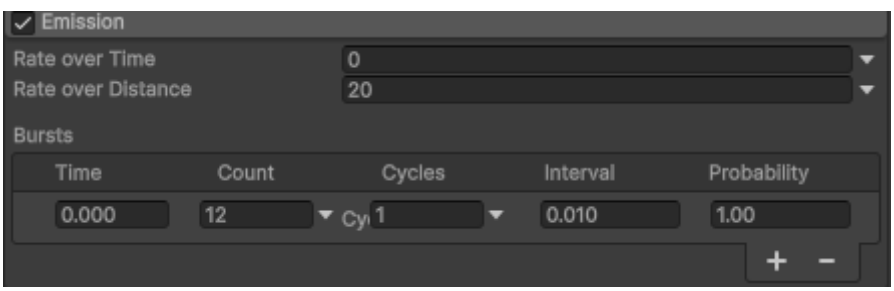
Smoke trail settings

The settings used for the smoke trail effect are as follows. All of these values can be copied, and there is plenty of room for customisation.

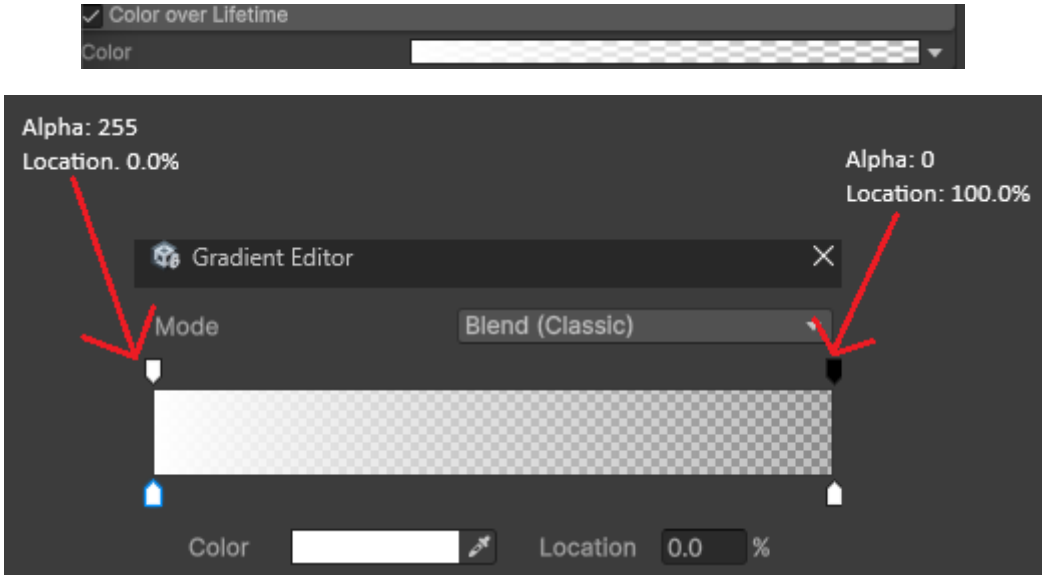
The main module of the Particle System should look like this:



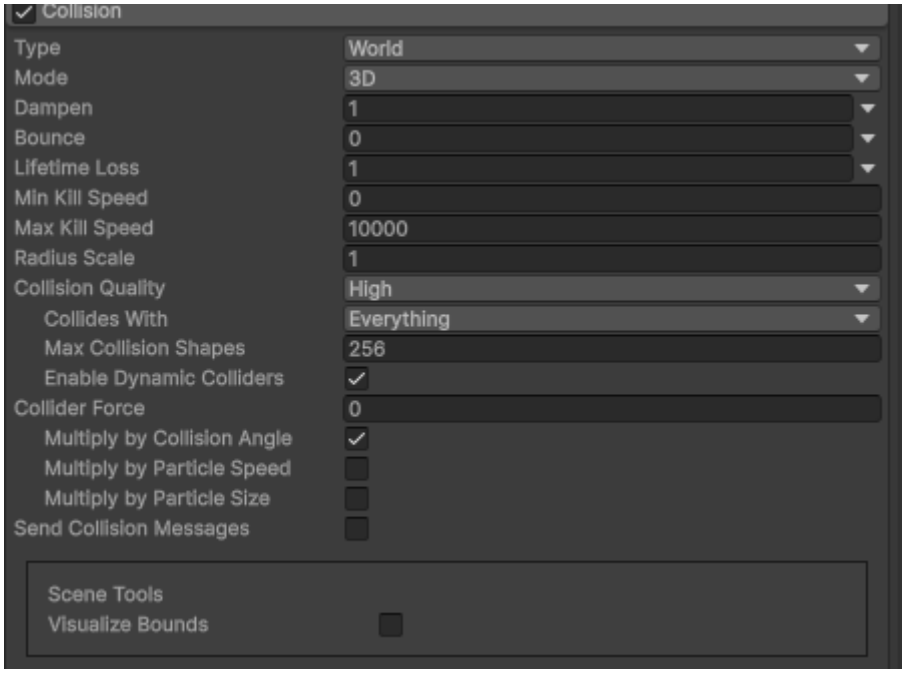
From there, the Emission box should be checked, with these values:



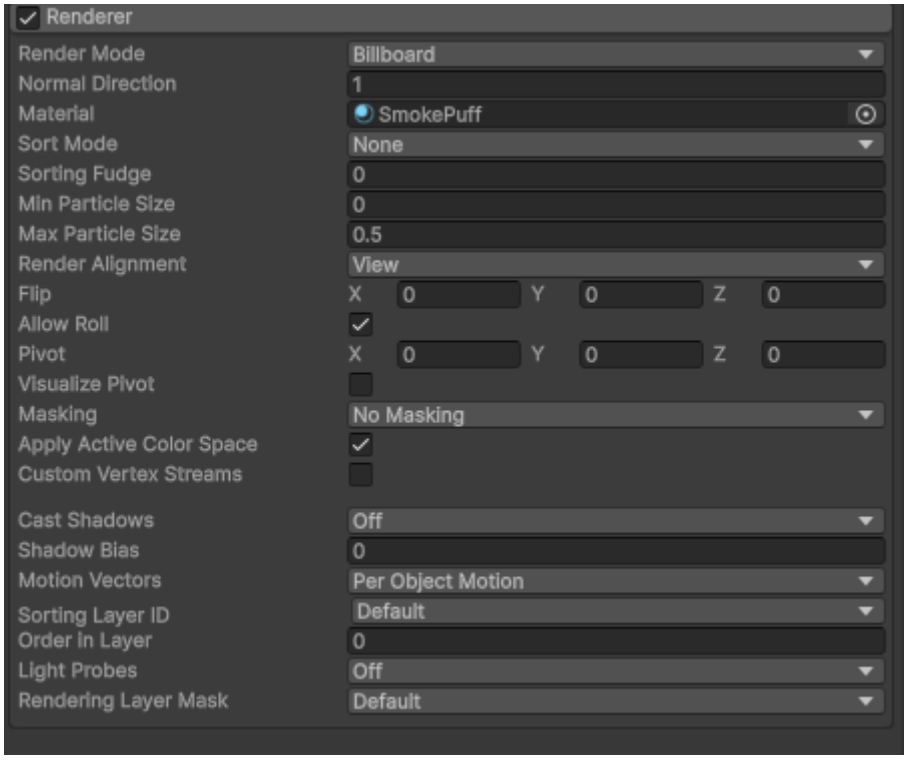
Next, Color over Lifetime module is used. Tick the box and left click on the colour bar to open the Gradient Editor and adjust it like so:



Scrolling down below Color over Lifetime module, the Collision module will be ticked. This module is useful to ensure that the smoke trail does not clip through objects or linger more than it should when at a very close distance to any collider or object.



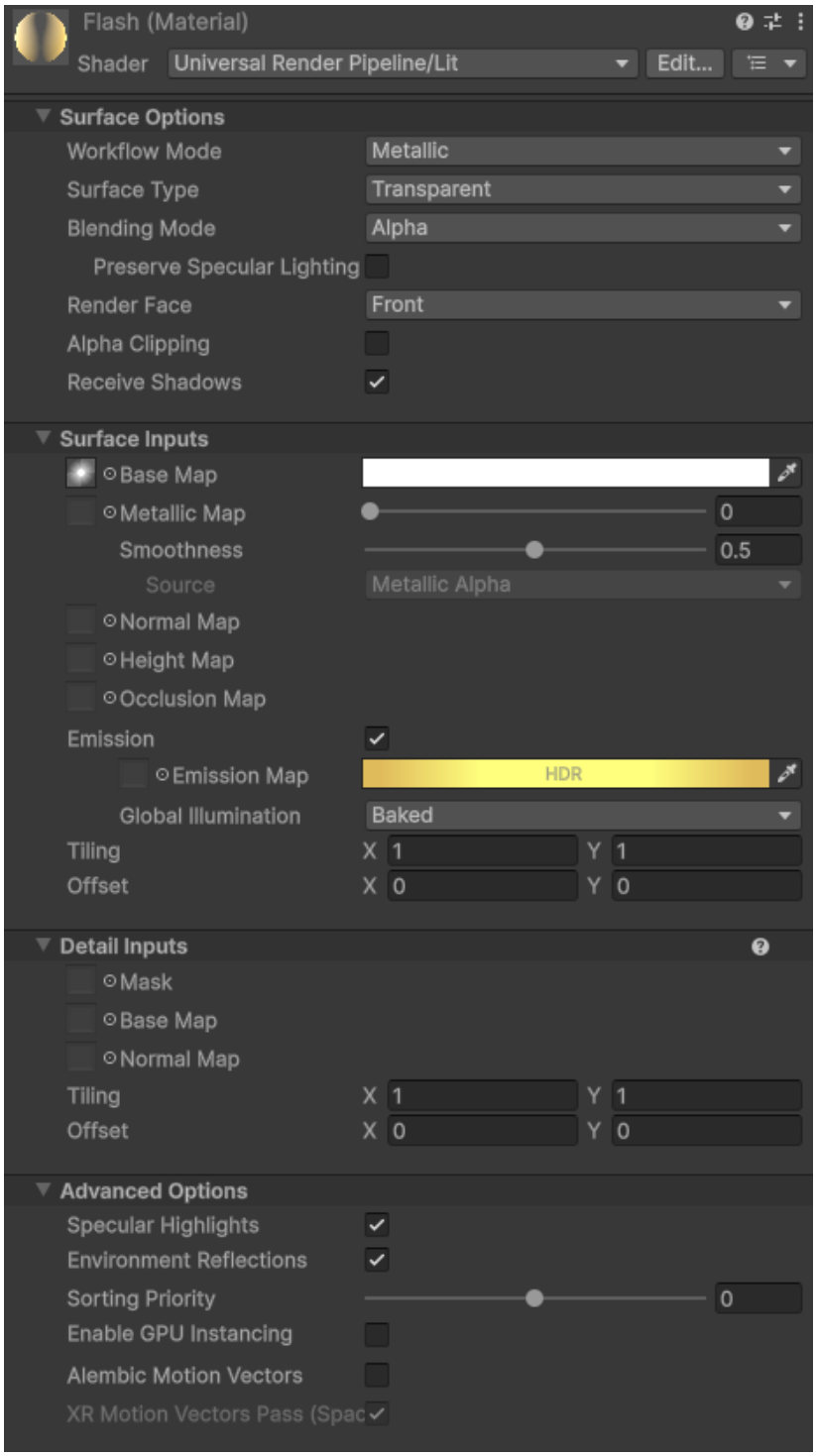
Finally, at the very bottom of the Particle System component there is the Renderer module which will need to be ticked.


Renderer
Render Mode
Billboard
Normal Direction
1
Material
SmokePuff
Sort Mode
None
Sorting Fudge
0
Min Particle Size
0
Max Particle Size
0.5
Render Alignment
View
Flip
X 0 Y 0 Z 0
Allow Roll
Pivot
X 0 Y 0 Z 0
Visualize Pivot
Masking
No Masking
Apply Active Color Space
Custom Vertex Streams
Cast Shadows
Off
Shadow Bias
0
Motion Vectors
Per Object Motion
Sorting Layer ID
Default
Order in Layer
0
Light Probes
Off
Rendering Layer Mask
Default

6. Muzzle Effects

Muzzle Flash

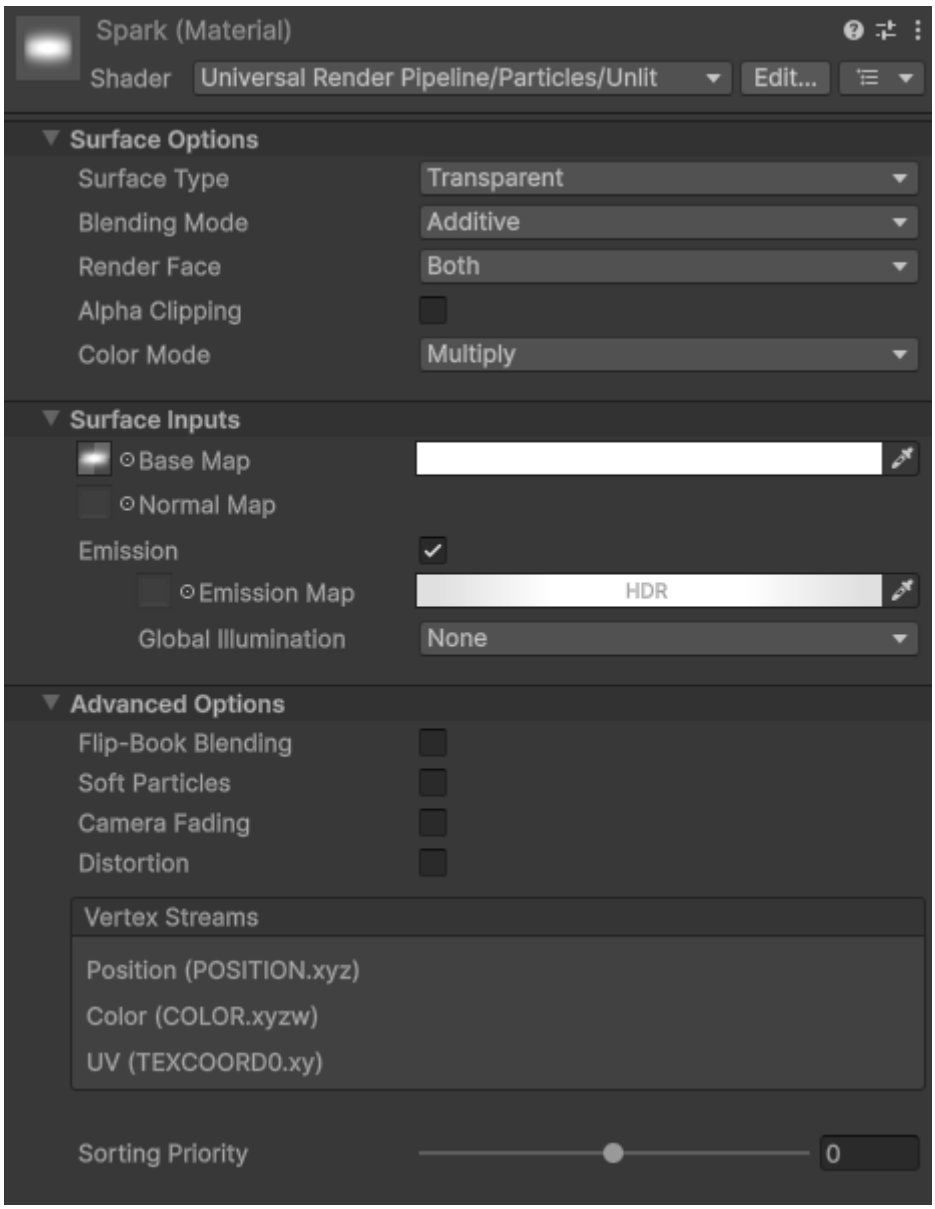
Secondary to this asset, there are three muzzle effects, with the primary being the muzzle flash. The material for the muzzle flash is located in the Materials folder and can simply be dragged into the muzzle flash slot of the object which contains the HitscanShooter script as a component.



The muzzle flash comes with a texture that is used in the Base Map slot of the flash material, but Unity's "Default-Particle" can be used instead.

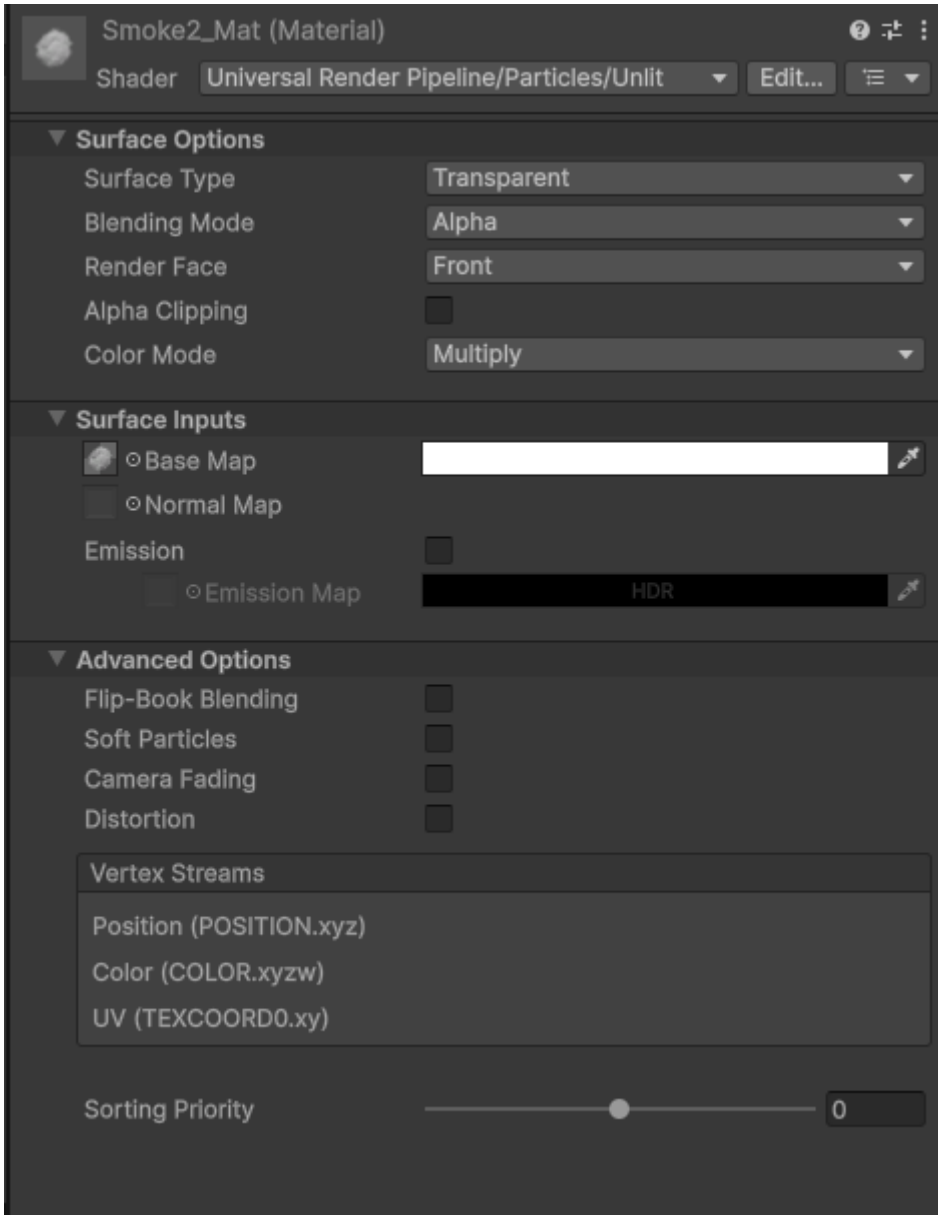
Muzzle Sparks

The muzzle sparks is a secondary effect which is basic and can be tuned significantly if desired. It uses a Particle System and is triggered in the HitscanShooter script per shot fired.



Muzzle Smoke

The muzzle smoke is the last effect in this asset where some slightly transparent and small puff of smoke occurs at the front of the gun per shot, called in the HitscanShooter script. There are several smoke texture variants which can be used for different visual appearances.



Different shader types can also be used if wanted. Overall, these effects were made to be simple but effective at what they do.

7. Troubleshooting

For any issues, support may be contacted here:
https://davem97.github.io/fpcc/contact